

3. Open / Closed Principle (OCP) in Java

3.1 Introduction

The **Open/Closed Principle (OCP)** is the second of the **SOLID** principles. It ensures that software entities (classes, modules, functions, etc.) are:

Open for extension but closed for modification.

3.2 Problem It Solves

Without OCP, every change (new logic, new rule, etc.) requires modifying existing code, risking:

- Bugs
- Regression
- Longer testing and deployment cycles

3.3 Definition of OCP

"Software entities should be open for extension, but closed for modification."

This means we should be able to add new functionality without touching existing code.

3.4 Real-World Analogy

If a printer supports printing to PDF, and we want to extend it to print to a network printer, we add new behavior instead of modifying existing printing logic.

3.5 Symptoms of OCP Violation

- Frequent changes to existing classes
- Long switch or if-else chains
- Adding a new type requires modifying multiple existing classes

3.6 Benefits of OCP

- Reduces bug surface
- Encourages modular, pluggable design
- Easier to extend and maintain

3.7 Implementation Guidelines

- Use interfaces and abstract classes
- Apply composition over inheritance
- Use patterns like Strategy, Command, Decorator

3.8 Examples

Example1: Violation: Discount Calculator

```
package com.mannemlokesh.violation.example1;

class DiscountCalculator {
```

```
        public double calculateDiscount(String customerType, double amount) {
            if (customerType.equals("REGULAR")) {
                return amount * 0.1;
            } else if (customerType.equals("PREMIUM")) {
                return amount * 0.2;
            }
            return 0;
        }
    }

    public class DiscountClient {
        public static void main(String[] args) {
            DiscountCalculator calc = new DiscountCalculator();
            System.out.println("Regular: " + calc.calculateDiscount("REGULAR",
                                                                    1000));
            System.out.println("Premium: " + calc.calculateDiscount("PREMIUM",
                                                                    1000));
        }
    }
}
```

Output:

```
Regular: 100.0
Premium: 200.0
```

Example1: Compliant : Discount Calculator

```
package com.mannemlokessh.compliant.example1;

interface DiscountStrategy {
    double getDiscount(double amount);
}

class RegularDiscount implements DiscountStrategy {
    public double getDiscount(double amount) {
        return amount * 0.1;
    }
}

class PremiumDiscount implements DiscountStrategy {
    public double getDiscount(double amount) {
        return amount * 0.2;
    }
}

class DiscountContext {
    private DiscountStrategy strategy;

    public DiscountContext(DiscountStrategy strategy) {
        this.strategy = strategy;
    }

    public double applyDiscount(double amount) {
        return strategy.getDiscount(amount);
    }
}

public class DiscountClientFixed {
    public static void main(String[] args) {
```

```
DiscountContext regular = new DiscountContext(new
RegularDiscount());
DiscountContext premium = new DiscountContext(new
PremiumDiscount());
System.out.println("Regular: " + regular.applyDiscount(1000));
System.out.println("Premium: " + premium.applyDiscount(1000));
    }
}
```

Output:

```
Regular: 100.0
Premium: 200.0
```

Example2: Violation: Shape Area Calculation

```
package com.mannemlokesh.violation.example2;

class AreaCalculator {
    public double calculateArea(Object shape) {
        if (shape instanceof Circle) {
            Circle c = (Circle) shape;
            return Math.PI * c.radius * c.radius;
        } else if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle) shape;
            return r.length * r.breadth;
        }
        return 0;
    }
}

class Circle {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }
}

class Rectangle {
    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }
}

public class ShapeAreaCalculation {
    public static void main(String[] args) {
        // ...
    }
}
```

Example2: Compliant : Shape Area Calculation

```
package com.mannemlokesh.compliant.example2;
```

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    public double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements Shape {
    double length, breadth;

    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }

    public double area() {
        return length * breadth;
    }
}

class AreaCalculator {
    public double calculate(Shape shape) {
        return shape.area();
    }
}

public class ShapeClient {
    public static void main(String[] args) {
        AreaCalculator calc = new AreaCalculator();
        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(4, 5);
        System.out.println("Circle Area: " + calc.calculate(circle));
        System.out.println("Rectangle Area: " + calc.calculate(rectangle));
    }
}
```

Output:

```
Circle Area: 78.53981633974483
Rectangle Area: 20.0
```

Example3: Complaint : Payment Processing Using Strategy

```
package com.mannemlokesh.compliant.example3;

interface PaymentStrategy {
    void pay(double amount);
}
```

```

}

class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid via Credit Card: " + amount);
    }
}

class UpiPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid via UPI: " + amount);
    }
}

class PaymentProcessor {
    private PaymentStrategy strategy;

    public PaymentProcessor(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void makePayment(double amount) {
        strategy.pay(amount);
    }
}

public class PaymentClient {
    public static void main(String[] args) {
        PaymentProcessor card = new PaymentProcessor(new
CreditCardPayment());
        PaymentProcessor upi = new PaymentProcessor(new UpiPayment());

        card.makePayment(500);
        upi.makePayment(700);
    }
}

```

Output:

```

Paid via Credit Card: 500.0
Paid via UPI: 700.0

```

3.9 Common Pitfalls

- Modifying existing logic for every new condition
- Misusing inheritance instead of composition
- Overusing reflection or instanceof

3.10 OCP vs LSP vs ISP

Principle	Focus
OCP	Don't modify, extend instead
LSP	Subclasses must be replaceable
ISP	Prefer many small interfaces

3.11 When to Apply OCP

- When adding new types or rules
- When code breaks due to minor requirement changes
- In plug-in style architectures

3.12 Summary

Feature	Description
Intent	Open for extension, closed for changes
Use Case	Calculators, processing rules, handlers
Key Benefit	Extensible, stable, flexible systems

The **Open/Closed Principle** makes your system more **modular and adaptable** to change — without breaking existing features.